

Alleva API Export vs. IZ Clinical Notes Analyzer

Deep Codebase Evaluation

Prepared for: R3 Recovery Services

Evaluation date: July 24, 2026

Scope: Read-only comparison of the supplied Alleva API export with the current V2 application code

Code changes: None

PHI-safe report: This document reports aggregate counts, schema coverage, and implementation behavior only. It does not reproduce patient names, patient identifiers, note text, credentials, signature images, or other record-level content from the workbook.

Executive verdict

The application's Alleva boundary is security-conscious and fail-closed, but the clinical import is not faithful to the API payload demonstrated by the workbook. It can safely identify and store treatment-plan records, yet it discards most nested clinical content, does not map the clinically relevant dates or signatures, never retrieves treatment reviews under the built-in contract, and can report a bounded partial pull as completed.

The implementation is therefore appropriate as a gated readiness prototype, but it is not currently reliable as a complete Alleva Treatment Plan Timeliness data source.

Evaluation area	Conclusion
Authentication and request protocol	Substantially correct
Patient/plan ownership validation	Correct and appropriately conservative
Encryption and versioned persistence	Substantially correct
Population completeness	Materially incomplete under default settings
Nested treatment-plan content fidelity	Materially incorrect and lossy
Plan, due-date, and signature semantics	Incorrect for the supplied runtime schema
Treatment-review retrieval	Functionally absent under the built-in contract
Job completion reporting	Can overstate completeness
Deterministic evaluation behavior	Correctly fail-closed, but starved by upstream evidence loss
Production-readiness claim	Not supported by this runtime evidence

What the workbook proves

The workbook's INCOMPLETE status does **not** mean that patient pagination, treatment-plan pagination, or detail retrieval failed.

- 365 unique patients were retrieved.
- There were no missing or duplicate patient IDs.
- 536 unique treatment plans were retrieved.
- There were no missing or duplicate treatment-plan IDs.
- All 536 detail requests succeeded.
- There were no detail failures and no detail retries.
- 532 plans mapped to a patient.
- Four plans could not be mapped back to a patient.
- Those four patient-mapping misses are the reason the export is marked INCOMPLETE.
- The omitted 126,223 long-form audit rows were a workbook-size optimization. All retrieved fields remain in the wide worksheets.

The exporter makes this distinction explicitly. Its completion test requires complete collections, no missing IDs, no mapping misses, no identity conflicts, and no detail failures:

```
diag-build-tools/Invoke-AllevaEndUserTools.ps1:2987
```

Long-form inclusion is calculated separately:

```
diag-build-tools/Invoke-AllevaEndUserTools.ps1:2991-3019
```

Runtime population

Population measure	Workbook result
Patients	365
Active patients	145
Discharged patients	218
Inactive patients	2
Treatment plans	536
Plans mapped to active patients	174
Plans mapped to discharged patients	358
Unmapped plans	4
Detail successes	536
Detail failures	0

Treatment-plan field coverage

- All 536 plans had `startDate`, `createdDate`, `lastModified`, `isActive`, `isComplete`, and `isInitialTP`.
- `PlanDate` equals `startDate` for all 536 plans.
- `PlanDate` equals `createdDate` for only 323 plans; 213 plans differ.
- All 536 plans were marked active in this snapshot.
- 513 plans were complete; 23 were incomplete.
- 154 plans were marked initial; 382 were not.
- Client-signature timestamps existed on 483 plans.
- Guardian-signature timestamps existed on 386 plans.
- No `nextReviewDue` field was present.
- No `staffSignatureDate` field was present.

Clinical-content richness

The API payload contained far more content than the application currently preserves:

Content type	Populated occurrences in export	Plans with content
Problem descriptions	631	536
Behavioral definitions	1,397	531
Diagnosis descriptions	1,065	536
Diagnosis codes	1,065	536
Goal descriptions	926	527
Objectives	2,024	527

Content type	Populated occurrences in export	Plans with content
Interventions	2,588	527

The list and detail payloads contained 2,000 paired field paths. For all 536 plans, every paired list/detail value was equal. In this tenant snapshot, the global `/treatment-plans` response already contained the complete detail payload; the subsequent 536 detail calls added no fields.

Findings ordered by severity

1. High - almost all nested clinical content is discarded

The runtime importer selects only the first dictionary encountered in each nested list:

```
backend/app/v2/services/alleva_sync.py:448-463
```

It also returns only the first non-empty diagnosis value:

```
backend/app/v2/services/alleva_sync.py:466-480
```

The aggregate builder maps those selected scalar values into `ParsedManualFields`:

```
backend/app/v2/services/alleva_sync.py:765-792
```

It then routes the API payload through the manual-upload aggregate builder. That builder constructs exactly one problem, one diagnosis, one behavioral definition, one goal, one objective, one intervention, and one signature:

```
backend/app/v2/services/manual_file_aggregate.py:92-130
```

Why this is incorrect: The domain schemas and frontend can represent arrays, but the API importer never populates those arrays. A plan containing multiple problems, goals, objectives, interventions, or diagnoses is reduced to the first discovered value in each category. The workbook demonstrates that this is not a theoretical edge case: it contains thousands of nested occurrences.

Observable result: The detail viewer can render multiple items, but API-imported records will normally display a single-item approximation rather than the actual treatment plan.

2. High - treatment-plan dates, due dates, and signatures are mapped incorrectly

The API aggregate uses:

- Patient admission date as the clock anchor.
- `nextReviewDue` as the plan due-date fallback.
- `staffSignatureDate` as the signature fallback.

```
backend/app/v2/services/alleva_sync.py:773-790
```

The supplied runtime schema has neither `nextReviewDue` nor `staffSignatureDate`. It does have `startDate`, `createdDate`, `lastModified`, `isInitialTP`, client signature timestamp, and guardian signature timestamp.

Those available fields are ignored. The manual builder then sets:

- `date_clock_anchor = admission_date`
- `date_clock_due_date = due_date`
- `source_due_date = due_date`

```
backend/app/v2/services/manual_file_aggregate.py:63-68
```

The database stores `aggregate.date_clock_anchor` in a column named `plan_date`:

```
backend/app/v2/services/treatment_plan_store.py:153-169
```

Why this is incorrect: In this runtime data, the clinical plan date is `startDate`; it is not the patient admission date. The application therefore persists an admission anchor under plan-date semantics while discarding the actual plan date.

The signature builder also assigns an unknown signer/evidence role:

```
backend/app/v2/services/manual_file_aggregate.py:218-226
```

The evaluator requires distinct `initial_plan` and `master_plan` signature roles:

```
backend/app/v2/services/evaluation_evidence.py:39-58
```

The overall evaluator returns `Missing Data` when initial signature, master signature, source due date, admission date, or plan evidence is missing:

```
backend/app/v2/services/deterministic_evaluator.py:98-118
```

Observable result: With the workbook's field shape, API-imported plans are driven into `Missing Data` even when Alleve supplied plan dates and client/guardian signature evidence. The evaluator is behaving conservatively; the upstream mapping is depriving it of usable evidence.

3. High - the default "full" pull is materially partial

The active V2 defaults are:

- API page size: 100
- Treatment-plan sync limit: 250

```
backend/app/v2/models.py:59-62
```

The same treatment-plan sync limit also caps the `/clients` collection:

```
backend/app/v2/services/alleve_contracts.py:118-123
```

Patients are capped **before** active-status filtering. If Alleve returns patients in the workbook's order, the default 250-record cap reaches:

- 250 of 365 patients
- 77 active patients
- 60 treatment plans associated with those active patients

At least 114 of the workbook's 174 active-patient plans are therefore unreachable under the default client cap.

The per-patient treatment-plan request then divides the global limit across active patients and explicitly stops after the first page:

```
backend/app/v2/services/alleve_sync.py:379-406
```

For those 77 active patients, the calculated per-patient page size is three. If Alleve honors the requested `Limit`, only 57 of those 60 reachable plans would be returned.

Why this is problematic: The active V2 contract says the importer traverses every bounded page for each active patient:

```
docs/v2-beta/api-contract-alleve.md:22
```

The implementation's `stop_after_first_page=True` contradicts that documented behavior.

The application also intentionally excludes:

- 358 plans belonging to discharged patients.
- Four plans without a trusted patient mapping.

Excluding unmapped plans is correct. Active-only scope can also be a legitimate operational decision. However, neither behavior represents a complete API export, and the UI label `Pull full treatment plans` overstates what the importer does:

```
frontend/src/v2/pages/TreatmentPlansPage.tsx:170
```

4. High - treatment reviews are declared but never retrieved

The built-in contract declares:

- `/treatment-reviews`
- `/treatment-reviews/{review_id}`

```
backend/app/v2/services/alleva_contracts.py:175-183
```

The detail loader only requests reviews when:

- The review endpoint path contains `{plan_id}`, or
- The review field mapping contains `plan_id`.

```
backend/app/v2/services/alleva_sync.py:680-697
```

The built-in review contract satisfies neither condition. Therefore, the review list and review-detail endpoints are never called during the operational importer.

What is correct: The code does not guess a relationship from `clientName` or another unsafe textual match. The documentation correctly states that Alleve has not established a trustworthy review foreign key.

What is incomplete: The contract and data model imply review support, but runtime behavior always leaves review evidence unrequested. The evaluator consequently has no signed-review anchor for recurring timeliness.

5. Medium-high - bounded and skipped work can be reported as successfully completed

The treatment-plan job is marked `completed_with_warnings` only when detail retrieval fails:

```
backend/app/v2/services/jobs.py:504-533
```

The following do not create a warning:

- Reaching the 250-client cap.
- Stopping after one plan page.
- Skipping a plan because its ID or ownership mapping is unusable.
- An active patient having no returned treatment plan.

The field named `imported_patient_count` is also not a patient count. It is the sum of created, updated, and unchanged treatment-plan records:

```
backend/app/v2/services/alleva_sync.py:198-205
```

The separate patient-roster job handles bounded snapshots more accurately by exposing `complete_snapshot` and a warning status:

```
backend/app/v2/services/jobs.py:535-595
```

Observable result: A treatment-plan job may show completed even though only a bounded subset of patients or plans was inspected. Dashboard counts and audit details can also label plan counts as patient counts.

6. Medium - "current plan" is selected by import timing, not clinical timing

Treatment-plan imports are ordered by the latest `imported_at` timestamp:

```
backend/app/v2/services/treatment_plan_store.py:47-66
```

The patient roster keeps the first row encountered for each patient/source pair:

```
backend/app/v2/services/patient_roster.py:24-43
```

The importer does not use:

- `startDate`
- `createdDate`
- `isInitialTP`
- `isComplete`
- A signed review date

to choose the clinically current plan.

Why this is problematic: During a bulk import, "current" becomes dependent on API response and database-save order. It is not a determination based on plan chronology or plan lifecycle.

7. Medium - encrypted checkpoints retain more PHI than documentation suggests

Every raw client and treatment-plan list page is serialized and written to `sync_checkpoints`:

```
backend/app/v2/services/alleva_contracts.py:356-378
```

The record payload is encrypted before persistence, which is correct. However:

- `/clients` returned direct identifiers and other patient demographics.
- In this tenant snapshot, `/treatment-plans` list rows contained the entire clinical detail payload and signature fields.
- No active V2 checkpoint cleanup or retention implementation was found.
- Resumed jobs copy the encrypted checkpoint records forward.

```
backend/app/v2/services/alleva_contracts.py:381-419
```

This conflicts with the V2 contract statement that patient names are not stored:

```
docs/v2-beta/api-contract-alleva.md:10
```

Assessment: Normalized roster and display records remain name-free, audit events avoid raw payloads, and encryption reduces disclosure risk. Nevertheless, encrypted checkpoint persistence is still storage. It needs an explicit retention and deletion boundary if the minimum-necessary claim is to remain accurate.

8. Medium - the synthetic tests validate an easier payload than the real export

The primary V2 sync mock:

- Supplies nested `client.id` and `client.route`.

- Supplies nextReviewDue, which the workbook lacks.
- Supplies only one problem, one goal, one objective, and one intervention.
- Ignores the requested plan Limit.
- Does not exercise actual client/guardian signatures, startDate, isInitialTP, multiple nested content items, or the workbook's plain unpagged collection behavior.

```
backend/tests/test_v2_alleva_sync.py:26-121
```

The test that verifies every returned plan passes partly because the mock returns all patient plans regardless of the request's Limit.

Validation result: A focused set of 33 Alleva protocol, sync, contract-runtime, and roster tests passed. There was one existing Starlette/httpx deprecation warning. The passing tests establish consistency with the synthetic fixtures; they do not establish fidelity with the supplied runtime schema.

9. Lower - detail calls were redundant in this export snapshot

The application fetches /treatment-plans/{id} for every validated plan:

```
backend/app/v2/services/alleva_sync.py:525-546
```

The workbook demonstrated that every one of the 2,000 list/detail field pairs matched for all 536 plans.

Assessment: The detail call may still be needed for other tenants or future API behavior, so this is not a general correctness defect. For this observed tenant behavior, however, the calls increased request volume, processing time, and the probability of detail-stage failures without adding data. The production contract should verify whether list/detail equivalence is stable before retaining or removing this step.

10. Unresolved contract risk - lower Limit and Cursor behavior is not proven

The exporter obtained complete plain-array responses by requesting Limit=5000 and observing a short, unpagged response. Its termination reason was unpagged_short_page.

The application assumes that lower Limit and Cursor values support conventional bounded paging. The workbook proves completeness for the 5,000-record request, but it does not prove that the endpoint honors smaller limits and cursors consistently.

Assessment: This is not a demonstrated code defect. It remains an unvalidated live-contract dependency and should not be inferred from synthetic wrapper-based tests.

Code that is correct or defensible

Exact protocol behavior

The request protocol correctly preserves:

- Case-sensitive ClientId
- StartDate
- Limit
- Cursor
- X-Version

- Bearer authorization
- Plain-list and common wrapper response shapes

```
backend/app/v2/services/alleva_protocol.py:7-68
```

Production and tenant gates

Live sync is disabled by default and requires:

- API testing enabled
- Treatment-plan sync enabled
- Explicit live-import authorization for the tenant
- Encrypted OAuth client configuration
- Trusted production origins in restricted environments

```
backend/app/v2/models.py:63-66
```

```
backend/app/v2/api/alleva_sync_routes.py:196-214
```

These are appropriate safeguards and align with the repository's gated Alleva boundary.

Ownership validation

The importer validates ownership from:

- Mapped nested client ID
- Mapped route/reference
- Raw scalar `/clients/{id}` reference
- Raw nested `client.id`
- Raw nested `client.route` or `client.href`

It rejects the plan if observed references are malformed, conflict, or do not all equal the queried patient:

```
backend/app/v2/services/alleva_sync.py:599-638
```

This behavior is correct. It would safely exclude the four workbook plans that lacked a trusted patient mapping.

Request-safety controls

The sync includes:

- Maximum row and response-size ceilings
- Repeated-page detection
- Retry limits for transient response codes
- Shared request-rate limiting
- Cancellation checks
- Bounded HTTP concurrency
- Encrypted resumable page checkpoints

```
backend/app/v2/services/alleva_sync.py:229-320
```

```
backend/app/v2/services/alleva_sync.py:722-741
```

Encrypted, versioned clinical persistence

Normalized aggregates are:

- Encrypted at rest

- Content-hashed
- Evidence-hashed
- Versioned rather than overwritten destructively
- Linked to sync and contract provenance
- Re-evaluated deterministically

```
backend/app/v2/services/treatment_plan_store.py:91-199
```

Minimum-necessary normalized UI

The normalized roster and ordinary UI exclude patient names and other unnecessary identifiers. The roster intentionally displays workflow identity rather than names:

```
frontend/src/v2/pages/PatientRosterPage.tsx:73-95
```

The frontend models and detail viewer are capable of rendering multiple problems, diagnoses, goals, objectives, interventions, and signatures. The loss occurs in backend normalization, not in the React presentation layer.

Fail-closed deterministic evaluation

The evaluator preserves:

- Missing Data
- Needs Review
- Conflicting Evidence
- Unable to Evaluate

It does not guess compliance when signatures, due dates, admission anchors, plan evidence, LOC mapping, or review evidence are absent or inconsistent.

```
backend/app/v2/services/deterministic_evaluator.py:98-118
```

That behavior is correct. The main defect is that the API normalization layer fails to carry available evidence into the evaluator's typed model.

API-to-application comparison

API/export capability	Current application behavior	Assessment
365-patient roster	Default treatment sync stops at 250 clients	Partial by default
Active, discharged, and inactive patients	Operational plan import processes active patients only	Intentional scope limit
536 total plans	At most a bounded active-patient subset	Not a full import
Four plans without trusted patient linkage	Rejected by ownership validation	Correct
Complete nested problems/goals/objectives/interventions	First item from each hierarchy only	Incorrect and lossy
Multiple diagnoses	First mapped diagnosis only	Incorrect and lossy
startDate on all plans	Ignored; admission date stored as plan date	Incorrect
isInitialTP	Ignored	Prevents plan-type classification
isComplete	Ignored	Loses lifecycle state
Client and guardian signature timestamps	Ignored	Evidence loss
No staffSignatureDate	App looks for staffSignatureDate	Mapping mismatch
No nextReviewDue	App looks for nextReviewDue	Mapping mismatch
Treatment-review endpoints	Declared but not invoked by built-in mapping	Functionally absent
List/detail payloads identical in snapshot	Detail fetched again for every plan	Redundant for observed behavior
Rich raw list payload	Encrypted in checkpoint database	Secure but retention-sensitive
Complete export reports mapping misses	Treatment sync warns only on detail failures	Completeness telemetry gap

Recommended correction order

This report is evaluative only; no code was changed. If implementation work is authorized later, the correction order should be:

- 1 Replace the manual scalar bridge with direct construction of the typed multi-item clinical snapshot.
- 2 Map and persist startDate, createdDate, isInitialTP, isComplete, and actual client/guardian signature metadata.
- 3 Define an evidence-role strategy that does not mislabel signatures and does not fabricate staff signatures.
- 4 Separate source document date, admission anchor, calculated recurrence date, and source-provided due date.
- 5 Remove stop_after_first_page=True or explicitly rename/document the operation as a bounded preview.
- 6 Surface client-cap, plan-cap, skipped-record, and incomplete-snapshot warnings in job status and audit counters.
- 7 Rename imported_patient_count to reflect that it counts treatment plans.
- 8 Select the current plan using validated clinical chronology and lifecycle fields, not import timestamp.
- 9 Resolve the review-to-plan relationship only with an approved stable ID contract; continue to refuse name-based joins.

- 10 Add runtime-shaped synthetic fixtures derived from the workbook schema, without copying PHI or record-level values.
 - 11 Establish explicit retention and cleanup policy for encrypted raw checkpoints.
 - 12 Verify live behavior for smaller Limit/Cursor values and list/detail equivalence before changing request strategy.
-

Validation performed

- Workbook inspected read-only using aggregate counts, schema paths, and completeness statistics.
 - No patient names, IDs, note text, credential values, or signature data were included in the analysis.
 - Current V2 sync, contract, persistence, evaluator, roster, job, and frontend paths were traced.
 - Focused automated validation:
 - `backend/tests/test_v2_alleva_protocol.py`
 - `backend/tests/test_v2_alleva_sync.py`
 - `backend/tests/test_v2_sync_contract_runtime.py`
 - `backend/tests/test_v2_roster_export.py`
 - Result: **33 passed**, with one existing Starlette/httpx deprecation warning.
 - No repository application code was modified.
-

Source references

Workbook analyzed

`C:\Users\r3developer\Downloads\exports\alleva-complete-export-20260715-224838-239-7a69a711-INCOMPLETE.xlsx`

Relevant ranges:

- Summary: A1:B29
- Patient Roster: A1:BE366
- Treatment Plans: A1:EXH537

Repository

`C:\Users\r3developer\OneDrive - R3 Recovery Services Inc\Development\IZ_clinical-notes-analyzer`

The evaluation reflects the current working tree observed on July 24, 2026. Pre-existing user changes in the working tree were preserved and not modified.

End of report